

算法板子

Contents

1. NTT	2
2. 计算几何	2
2.1. 距离的转换	2
2.2. pick 定理	2
2.3. 凸包 (andrew 算法)	2
2.4. 扫描线	2
2.5. 旋转卡壳	2
2.6. 半平面交	2
2.7. 随机增量法	3
2.8. 反演变换	3
2.9. Delaunay 三角剖分	5
2.10. 二、三维计算几何	6
2.11. 仿射变换	6
3. 数论	6
3.1. Exgcd	6
3.2. Bsgs ($\gcd(a,p)=1$)	6
3.3. ExCRT (扩展中国剩余定理)	6
3.4. 拓展欧拉定理	6
4. 欧拉路径	6
5. 马拉车	6
6. 欧拉筛	6
7. 拉格朗日插值 (通用版, $O(n^2)$)	6
8. 拉格朗日插值 (连续整数)	7
9. Odt	7
10. 前缀数组 (来自 oiwiki)	7
11. kmp (来自 oiwiki)	7
12. mt19937	7
13. 二分图最大匹配 (匈牙利算法, 稠密图且点数大于 $1e4$ 不适用)	7
14. 线性基	7
15. 强连通分量	8
16. polya 定理	8
17. 求割点	8
18. 自定义 hash	8
19. hall 定理	8
20. Zobrist Hashing (xor hashing)	8
21. Lemma (Bertrand's postulate)	9
22. 求多个数的欧拉函数	9
23. 最大流 (dinic 算法)	9
24. 有源汇最大流	9
25. 最小费用最大流 (spfa)	9
26. 开区间二分(l,r)	10
27. 线性求逆元	10
28. 数位 dp	10
29. 关于 gcd 的常数优化	10
30. 高精度	11
31. 闭区间线段树	11
32. 杂项	12
32.1. sos dp	12

1. NTT

```
const int MOD = 998244353, G = 3; // 常见 NTT 模数及其原根
// power 是 ksm
void ntt(vector<int>& a, bool inv) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n > 1;
        for (; j & bit; bit >= 1) j = bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        ll wlen = power(G, (MOD - 1) / len);
        if (inv) wlen = power(wlen, MOD - 2);
        for (int i = 0; i < n; i += len) {
            ll w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i + j], v = (int)(1LL * a[i + j + len / 2] * w % MOD);
                a[i + j] = (u + v) % MOD;
                a[i + j + len / 2] = (u - v + MOD) % MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (inv) {
        ll n_inv = power(n, MOD - 2);
        for (int& x : a) x = (int)(1LL * x * n_inv % MOD);
    }
}
vector<int> multiply(vector<int> a, vector<int> b) {
    int n = 1, total = a.size() + b.size() - 1;
    while (n < total) n <= 1;
    a.resize(n); b.resize(n);
    ntt(a, 0); ntt(b, 0);
    for (int i = 0; i < n; i++) a[i] = 1LL * a[i] * b[i] % MOD;
    ntt(a, 1);
    a.resize(total);
    // 求循环卷积 return res
    /* vector<int> res(L, 0);
    for (int i = 0; i < n; i++) {
        res[i % L] = (res[i % L] + a[i]) % MOD;
    } */
    return a;
}
```

2. 计算几何

2.1. 距离的转换

$$|x_1 - x_2| + |y_1 - y_2| = \max(|(x_1 + y_1) - (x_2 + y_2)|, |(x_1 - y_1) - (x_2 - y_2)|)$$

2.2. pick 定理

给定顶点均为整点的简单多边形，皮克定理说明了其面积 A 和内部格点数目 i 、边上格点数目 b 的关系

$$A = i + \frac{b}{2} - 1$$

2.3. 凸包 (andrew 算法)

```
struct Point {
    long long x, y;
    // 排序: 先按 x 升序, 再按 y 升序
    bool operator<(const Point& t) const {
        return x == t.x ? y < t.y : x < t.x;
    }
};
// 叉积: (B-A) x (C-A)。>0 表示 C 在 AB 左侧 (逆时针旋转)
long long cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}
vector<Point> getConvexHull(vector<Point>& p) {
    int n = p.size(), k = 0;
    if (n <= 2) return p;
    vector<Point> res(2 * n);
    sort(p.begin(), p.end());
    // 求下凸壳
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(res[k - 2], res[k - 1], p[i]) <= 0) k--;
        res[k++] = p[i];
    }
    // 求上凸壳
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross(res[k - 2], res[k - 1], p[i]) <= 0) k--;
        res[k++] = p[i];
    }
    res.resize(k - 1); // 最后一个点和第一个点重合, 删掉
    return res;
}
```

2.3.1. 闵可夫斯基和

```
template <class T>
struct Point {
    T x, y;

    Point(T x = 0, T y = 0) : x(x), y(y) {}

    friend Point operator+(const Point &a, const Point &b) {
        return {a.x + b.x, a.y + b.y};
    }

    friend Point operator-(const Point &a, const Point &b) {
        return {a.x - b.x, a.y - b.y};
    }

    // 点乘
    friend T operator*(const Point &a, const Point &b) {
        return a.x * b.x + a.y * b.y;
    }

    // 叉乘
    friend T operator^(const Point &a, const Point &b) {
        return a.x * b.y - a.y * b.x;
    }
};

template <class T>
vector<Point<T>> minkowski_sum(vector<Point<T>> a, vector<Point<T>> b) {
    vector<Point<T>> c{a[0] + b[0]};
    for (usz i = 0; i + 1 < a.size(); ++i) a[i] = a[i + 1] - a[i];
    for (usz i = 0; i + 1 < b.size(); ++i) b[i] = b[i + 1] - b[i];
    a.pop_back(), b.pop_back();
    c.resize(a.size() + b.size() + 1);
    merge(a.begin(), a.end(), b.begin(), b.end(), c.begin() + 1,
        [](const Point<T> &a, const Point<T> &b) { return (a ^ b) > 0; });
    for (usz i = 1; i < c.size(); ++i) c[i] = c[i] + c[i - 1];
    return c;
}
```

2.4. 扫描线

2.5. 旋转卡壳

```
int sta[N], top; // 将凸包上的节点编号存在栈里, 第一个和最后一个节点编号相同

ll pf(ll x) { return x * x; }

ll dis(int p, int q) { return pf(a[p].x - a[q].x) + pf(a[p].y - a[q].y); }

ll sqr(int p, int q, int y) { return abs((a[q] - a[p]) * (a[y] - a[q])); }

ll mx;

void get_longest() { // 求凸包直径
    int j = 3;
    if (top < 4) {
        mx = dis(sta[1], sta[2]);
        return;
    }
    for (int i = 1; i < top; ++i) {
        while (sqr(sta[i], sta[i + 1], sta[j]) <=
            sqr(sta[i], sta[i + 1], sta[j % top + 1]))
            j = j % top + 1;
        mx = max(mx, max(dis(sta[i + 1], sta[j]), dis(sta[i], sta[j])));
    }
}
```

2.6. 半平面交

```
friend bool operator<(seg x, seg y) {
    db t1 = atan2((x.b - x.a).y, (x.b - x.a).x);
    db t2 = atan2((y.b - y.a).y, (y.b - y.a).x); // 求极角
    if (fabs(t1 - t2) > eps) // 如果极角不等
        return t1 < t2;
    return (y.a - x.a) * (y.b - x.a) >
        eps; // 判断向量 x 在 y 的哪边, 令最靠左的排在最左边
}
// pnt its(seg a, seg b) 表示求线段 a, b 的交点
// s[] 是极角排序后的向量
// q[] 是向量队列
// t[i] 是 s[i-1] 与 s[i] 的交点
// 【码风】队列的范围是 [l, r]
// 求的是向量左侧的半平面
int l = 0, r = 0;
for (int i = 1; i <= n; ++i)
    if (s[i] != s[i - 1]) {
        // 注意要先检查队尾
        while (r - l > 1 && (s[i].b - t[r]) * (s[i].a - t[r]) >
            eps) // 如果上一个交点在向量右侧则弹出队尾
            r--;
        while (l < r && (s[i].b - t[l]) * (s[i].a - t[l]) >
            eps) // 如果下一个交点在向量右侧则弹出队尾
            l++;
        t[l] = s[i];
        q.push_back(s[i]);
    }
}
```

```
--r;
while (r - l > 1 && (s[i].b - t[l + 2]) * (s[i].a - t[l + 2]) >
      eps) // 如果第一个交点在向量右侧则弹出队首
    ++l;
q[++r] = s[i];
if (r - l > 1) t[r] = its(q[r], q[r - 1]); // 求新交点
}
while (r - l > 1 &&
      (q[l + 1].b - t[r]) * (q[l + 1].a - t[r]) > eps) // 注意删除多余元素
    --r;
t[r + 1] = its(q[l + 1], q[r]); // 再求出新的交点
++r;
// 这里不能在t里面++r需要注意一下.....
```

2.7. 随机增量法

```
/*
最小圆覆盖问题: 在一个平面上有 n 个点, 求一个半径最小的圆, 能覆盖所有的点.
下面这个做法用随机化打乱后期望 o(n), 证明较为复杂, 下面给了两种写法(其实一样), 第二种是我自己写的, 感觉代码会更清晰点
*/
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <iostream>

using namespace std;

int n;
double r;

struct point {
    double x, y;
} p[10005], o;

double sqr(double x) { return x * x; }

double dis(point a, point b) { return sqrt(sqr(a.x - b.x) + sqr(a.y - b.y)); }

bool cmp(double a, double b) { return fabs(a - b) < 1e-8; }

point geto(point a, point b, point c) {
    double a1, a2, b1, b2, c1, c2;
    point ans;
    a1 = 2 * (b.x - a.x), b1 = 2 * (b.y - a.y);
    c1 = sqr(b.x) - sqr(a.x) + sqr(b.y) - sqr(a.y);
    a2 = 2 * (c.x - a.x), b2 = 2 * (c.y - a.y);
    c2 = sqr(c.x) - sqr(a.x) + sqr(c.y) - sqr(a.y);
    if (cmp(a1, 0)) {
        ans.y = c1 / b1;
        ans.x = (c2 - ans.y * b2) / a2;
    } else if (cmp(b1, 0)) {
        ans.x = c1 / a1;
        ans.y = (c2 - ans.x * a2) / b2;
    } else {
        ans.x = (c2 * b1 - c1 * b2) / (a2 * b1 - a1 * b2);
        ans.y = (c2 * a1 - c1 * a2) / (b2 * a1 - b1 * a2);
    }
    return ans;
}

int main() {
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) scanf("%lf%lf", &p[i].x, &p[i].y);
    for (int i = 1; i <= n; i++) swap(p[rand() % n + 1], p[rand() % n + 1]);
    o = p[1];
    for (int i = 1; i <= n; i++) {
        if (dis(o, p[i]) < r || cmp(dis(o, p[i]), r)) continue;
        o.x = (p[i].x + p[1].x) / 2;
        o.y = (p[i].y + p[1].y) / 2;
        r = dis(p[i], p[1]) / 2;
        for (int j = 2; j < i; j++) {
            if (dis(o, p[j]) < r || cmp(dis(o, p[j]), r)) continue;
            o.x = (p[i].x + p[j].x) / 2;
            o.y = (p[i].y + p[j].y) / 2;
            r = dis(p[i], p[j]) / 2;
            for (int k = 1; k < j; k++) {
                if (dis(o, p[k]) < r || cmp(dis(o, p[k]), r)) continue;
                o = geto(p[i], p[j], p[k]);
                r = dis(o, p[i]);
            }
        }
    }
    printf("%.10lf\n%.10lf %.10lf", r, o.x, o.y);
    return 0;
}
```

```
/*
自己写的最小圆覆盖代码, 有点丑
*/
```

```
#include <bits/stdc++.h>
using namespace std;
const double eps = 1e-9;
mt19937 rng(chrono::steady_clock::now(), time_since_epoch().count());
void solve() {
    int n;
    cin >> n;
    vector<pair<double, double>> a(n);
    for (auto &[x, y] : a) {
        cin >> x >> y;
    }
    shuffle(a.begin(), a.end(), rng);
    pair<double, double> c = a[0];
    double r = 0;
    auto dis = [&](pair<double, double> &a) -> double {
        auto &[x, y] = a;
        auto &[x1, y1] = c;
        return sqrt((x - x1) * (x - x1) + (y - y1) * (y - y1));
    };
    auto get_v = [&](vector<array<double, 2>> &x) -> double {
        auto [a1, b1] = x[0];
        auto [a2, b2] = x[1];
        return a1 * b2 - a2 * b1;
    };
    auto get_c = [&](int i, int j, int k) -> pair<double, double> {
        auto &[xi, yi] = a[i];
        auto &[xj, yj] = a[j];
        auto &[xk, yk] = a[k];
        vector<array<double, 2>> mk(2), mk_x(2), mk_y(2);
        mk[0] = {-2 * xi + 2 * xj, -2 * yi + 2 * yj};
        mk[1] = {-2 * xi + 2 * xk, -2 * yi + 2 * yk};
        double base = get_v(mk);
        mk_x[0] = {xj * xj - xi * xi + yj * yj - yi * yi, -2 * yi + 2 * yj};
        mk_x[1] = {xk * xk - xi * xi + yk * yk - yi * yi, -2 * yi + 2 * yk};
        double x = get_v(mk_x) / base;
        mk_y[0] = {-2 * xi + 2 * xj, xj * xj - xi * xi + yj * yj - yi * yi};
        mk_y[1] = {-2 * xi + 2 * xk, xk * xk - xi * xi + yk * yk - yi * yi};
        double y = get_v(mk_y) / base;
        return {x, y};
    };
    for (int i = 1; i < n; i++) {
        if (dis(a[i]) > r + eps) {
            c = a[i];
            r = 0;
            for (int j = 0; j < i; j++) {
                if (dis(a[j]) > r + eps) {
                    auto [x, y] = a[i];
                    auto [x1, y1] = a[j];
                    c = pair<double, double>{(x + x1) / 2, (y + y1) / 2};
                    r = dis(a[j]);
                    for (int k = 0; k < j; k++) {
                        if (dis(a[k]) > r + eps) {
                            auto [x1, y1] = pair<double, double>{a[k].first - a[i].first, a[k].second - a[i].second};
                            auto [x2, y2] = pair<double, double>{a[j].first - a[i].first, a[j].second - a[i].second};
                            if (fabs(x1 * y2 - x2 * y1) < eps) continue;
                            c = get_c(i, j, k);
                            r = dis(a[k]);
                        }
                    }
                }
            }
        }
    }
    cout << fixed << setprecision(9) << r << "\n" << c.first << " " << c.second << "\n";
    return;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
    return 0;
}
```

2.8. 反演变换

定义:

给定反演中心点 O 和反演半径 R , 若平面上点 P 和 P' 满足:

(1) 点 P' 在射线 OP 上

(2) $|OP| \cdot |OP'| = R^2$

则称点 P 和点 P' 互为反演点.

性质:

(1) 除反演中心外, 平面上的每一个点都只有唯一的反演点, 且这种关系是对称的, 位于反演圆上的点, 保持在那里, 位于反演圆外部的点, 变为圆内部的点, 位于反演圆内部的点, 变为圆外部的点.

(2) 任意一条不过反演中心的直线, 它的反形是经过反演中心的圆, 反之亦然, 特别地, 过反演中心相交的圆, 变为不过反演中心的相交直线

(3) 反演不改变相切、平行、相交的关系

记圆 A 半径为 r_1 , 其反演图形圆 B 半径为 r_2 , 则有:

$$r_2 = \frac{1}{2} \left(\frac{1}{|OA| - r_1} - \frac{1}{|OA| + r_1} \right) R^2$$

记点 O 坐标为 (x_0, y_0) , 点 A 坐标为 (x_1, y_1) , 点 B 坐标为 (x_2, y_2) , 则有:

$$x_2 = x_0 + \frac{|OB|}{|OA|} (x_1 - x_0)$$

$$y_2 = y_0 + \frac{|OB|}{|OA|} (y_1 - y_0)$$

```
/*
题目:求过两圆外一点, 且与两圆相切的所有的圆.
*/
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstring>
#include <iostream>
#include <vector>
using namespace std;

constexpr double EPS = 1e-8; // 精度系数
const double PI = acos(-1.0); // π
constexpr int N = 4;

// 点的定义
struct Point {
    double x, y;

    Point(double x = 0, double y = 0) : x(x), y(y) {}

    bool operator<(Point A) const { return x == A.x ? y < A.y : x < A.x; }
};

// 向量的定义
using Vector = Point;

// 向量加法
Vector operator+(Vector A, Vector B) { return Vector(A.x + B.x, A.y + B.y); }

// 向量减法
Vector operator-(Vector A, Vector B) { return Vector(A.x - B.x, A.y - B.y); }

// 向量数乘
Vector operator*(Vector A, double p) { return Vector(A.x * p, A.y * p); }

// 向量数除
Vector operator/(Vector A, double p) { return Vector(A.x / p, A.y / p); }

// 与0的关系
int dcmp(double x) {
    if (fabs(x) < EPS) return 0;
    return x < 0 ? -1 : 1;
}

// 向量点乘
double Dot(Vector A, Vector B) { return A.x * B.x + A.y * B.y; }

// 向量长度
double Length(Vector A) { return sqrt(Dot(A, A)); }

// 向量叉乘
double Cross(Vector A, Vector B) { return A.x * B.y - A.y * B.x; }

// 点在直线上投影
Point GetLineProjection(Point P, Point A, Point B) {
    Vector v = B - A;
    return A + v * (Dot(v, P - A) / Dot(v, v));
}

// 圆
struct Circle {
    Point c;
    double r;

    Circle() : c(Point(0, 0)), r(0) {}

    Circle(Point c, double r = 0) : c(c), r(r) {}

    // 输入极角返回点坐标
    Point point(double a) { return Point(c.x + cos(a) * r, c.y + sin(a) * r); }
};

// 两圆公切线 返回切线的条数, -1表示无穷多条切线
// a[i] 和 b[i] 分别是第i条切线在圆A和圆B上的切点
int getTangents(Circle A, Circle B, Point* a, Point* b) {
    int cnt = 0;
    if (A.r < B.r) {
        swap(A, B);
    }
}
```

```
swap(a, b);
}

double d2 =
    (A.c.x - B.c.x) * (A.c.x - B.c.x) + (A.c.y - B.c.y) * (A.c.y - B.c.y);
double rdif = A.r - B.r;
double rsum = A.r + B.r;
if (dcmp(d2 - rdif * rdif) < 0) return 0; // 内含

double base = atan2(B.c.y - A.c.y, B.c.x - A.c.x);
if (dcmp(d2) == 0 && dcmp(A.r - B.r) == 0) return -1; // 无限多条切线
if (dcmp(d2 - rdif * rdif) == 0) { // 内切, 一条切线
    a[cnt] = A.point(base);
    b[cnt] = B.point(base);
    ++cnt;
    return 1;
}

// 有外公切线
double ang = acos(rdif / sqrt(d2));
a[cnt] = A.point(base + ang);
b[cnt] = B.point(base + ang);
++cnt;
a[cnt] = A.point(base - ang);
b[cnt] = B.point(base - ang);
++cnt;
if (dcmp(d2 - rsum * rsum) == 0) { // 一条内公切线
    a[cnt] = A.point(base);
    b[cnt] = B.point(PI + base);
    ++cnt;
} else if (dcmp(d2 - rsum * rsum) > 0) { // 两条内公切线
    double ang = acos(rsum / sqrt(d2));
    a[cnt] = A.point(base + ang);
    b[cnt] = B.point(PI + base + ang);
    ++cnt;
    a[cnt] = A.point(base - ang);
    b[cnt] = B.point(PI + base - ang);
    ++cnt;
}
return cnt;
}

// 点O在圆A外, 求圆A的反演圆B, R是反演半径
Circle Inversion_C2C(Point O, double R, Circle A) {
    double OA = Length(A.c - O);
    double RB = 0.5 * ((1 / (OA - A.r)) - (1 / (OA + A.r))) * R * R;
    double OB = OA * RB / A.r;
    double Bx = O.x + (A.c.x - O.x) * OB / OA;
    double By = O.y + (A.c.y - O.y) * OB / OA;
    return Circle(Point(Bx, By), RB);
}

// 直线反演为过O点的圆B, R是反演半径
Circle Inversion_L2C(Point O, double R, Point A, Vector v) {
    Point P = GetLineProjection(O, A, A + v);
    double d = Length(O - P);
    double RB = R * R / (2 * d);
    Vector VB = (P - O) / d * RB;
    return Circle(O + VB, RB);
}

// 返回 true 如果 A B 两点在直线同侧
bool theSameSideOfLine(Point A, Point B, Point S, Vector v) {
    return dcmp(Cross(A - S, v)) * dcmp(Cross(B - S, v)) > 0;
}

int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        Circle A, B;
        Point P;
        scanf("%lf%lf", &A.c.x, &A.c.y, &A.r);
        scanf("%lf%lf", &B.c.x, &B.c.y, &B.r);
        scanf("%lf%lf", &P.x, &P.y);
        Circle NA = Inversion_C2C(P, 10, A);
        Circle NB = Inversion_C2C(P, 10, B);
        Point LA[N], LB[N];
        Circle ansC[N];
        int q = getTangents(NA, NB, LA, LB), ans = 0;
        for (int i = 0; i < q; ++i)
            if (theSameSideOfLine(NA.c, NB.c, LA[i], LB[i] - LA[i])) {
                if (!theSameSideOfLine(P, NA.c, LA[i], LB[i] - LA[i])) continue;
                ansC[ans++] = Inversion_L2C(P, 10, LA[i], LB[i] - LA[i]);
            }
        printf("%d\n", ans);
        for (int i = 0; i < ans; ++i) {
            printf("%.8f %.8f %.8f\n", ansC[i].c.x, ansC[i].c.y, ansC[i].r);
        }
    }

    return 0;
}
```

2.9. Delaunay 三角剖分

```
#include <algorithm>
#include <cmath>
#include <cstring>
#include <list>
#include <utility>
#include <vector>

constexpr double EPS = 1e-8;
constexpr int MAXV = 10000;

struct Point {
    double x, y;
    int id;

    Point(double a = 0, double b = 0, int c = -1) : x(a), y(b), id(c) {}

    bool operator<(const Point &a) const {
        return x < a.x || (fabs(x - a.x) < EPS && y < a.y);
    }

    bool operator==(const Point &a) const {
        return fabs(x - a.x) < EPS && fabs(y - a.y) < EPS;
    }

    double dist2(const Point &b) {
        return (x - b.x) * (x - b.x) + (y - b.y) * (y - b.y);
    }
};

struct Point3D {
    double x, y, z;

    Point3D(double a = 0, double b = 0, double c = 0) : x(a), y(b), z(c) {}

    Point3D(const Point &p) { x = p.x, y = p.y, z = p.x * p.x + p.y * p.y; }

    Point3D operator-(const Point3D &a) const {
        return Point3D(x - a.x, y - a.y, z - a.z);
    }

    double dot(const Point3D &a) { return x * a.x + y * a.y + z * a.z; }
};

struct Edge {
    int id;
    std::list<Edge>::iterator c;

    Edge(int id = 0) { this->id = id; }
};

int cmp(double v) { return fabs(v) > EPS ? (v > 0 ? 1 : -1) : 0; }

double cross(const Point &o, const Point &a, const Point &b) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

Point3D cross(const Point3D &a, const Point3D &b) {
    return Point3D(a.y * b.z - a.z * b.y, -a.x * b.z + a.z * b.x,
        a.x * b.y - a.y * b.x);
}

int inCircle(const Point &a, Point b, Point c, const Point &p) {
    if (cross(a, b, c) < 0) std::swap(b, c);
    Point3D a3(a), b3(b), c3(c), p3(p);
    b3 = b3 - a3, c3 = c3 - a3, p3 = p3 - a3;
    Point3D f = cross(b3, c3);
    return cmp(p3.dot(f)); // check same direction, in: < 0, on: = 0, out: > 0
}

int intersection(const Point &a, const Point &b, const Point &c,
    const Point &d) { // seg(a, b) and seg(c, d)
    return cmp(cross(a, c, b)) * cmp(cross(a, b, d)) > 0 &&
        cmp(cross(c, a, d)) * cmp(cross(c, d, b)) > 0;
}

class Delaunay {
public:
    std::list<Edge> head[MAXV]; // graph
    Point p[MAXV];
    int n, rename[MAXV];

    void init(int n, Point p[]) {
        memcpy(this->p, p, sizeof(Point) * n);
        std::sort(this->p, this->p + n);
        for (int i = 0; i < n; i++) rename[p[i].id] = i;
        this->n = n;
        divide(0, n - 1);
    }

    void addEdge(int u, int v) {
        head[u].push_front(Edge(v));
        head[v].push_front(Edge(u));
        head[u].begin()->c = head[v].begin();
        head[v].begin()->c = head[u].begin();
    }
};
```

```
void divide(int l, int r) {
    if (r - l <= 2) { // #point <= 3
        for (int i = l; i <= r; i++)
            for (int j = i + 1; j <= r; j++) addEdge(i, j);
        return;
    }
    int mid = (l + r) / 2;
    divide(l, mid);
    divide(mid + 1, r);

    std::list<Edge>::iterator it;
    int nowl = l, nowr = r;

    for (int update = 1; update;) {
        // find left and right convex, lower common tangent
        update = 0;
        Point ptL = p[nowl], ptR = p[nowr];
        for (it = head[nowl].begin(); it != head[nowl].end(); it++) {
            Point t = p[it->id];
            double v = cross(ptR, ptL, t);
            if (cmp(v) > 0 || (cmp(v) == 0 && ptR.dist2(t) < ptR.dist2(ptL))) {
                nowl = it->id, update = 1;
                break;
            }
        }
        if (update) continue;
        for (it = head[nowr].begin(); it != head[nowr].end(); it++) {
            Point t = p[it->id];
            double v = cross(ptL, ptR, t);
            if (cmp(v) < 0 || (cmp(v) == 0 && ptL.dist2(t) < ptL.dist2(ptR))) {
                nowr = it->id, update = 1;
                break;
            }
        }
    }

    addEdge(nowl, nowr); // add tangent

    for (int update = 1; true;) {
        update = 0;
        Point ptL = p[nowl], ptR = p[nowr];
        int ch = -1, side = 0;
        for (it = head[nowl].begin(); it != head[nowl].end(); it++) {
            if (cmp(cross(ptL, ptR, p[it->id])) > 0 &&
                (ch == -1 || inCircle(ptL, ptR, p[ch], p[it->id]) < 0)) {
                ch = it->id, side = -1;
            }
        }
        for (it = head[nowr].begin(); it != head[nowr].end(); it++) {
            if (cmp(cross(ptR, p[it->id], ptL)) > 0 &&
                (ch == -1 || inCircle(ptL, ptR, p[ch], p[it->id]) < 0)) {
                ch = it->id, side = 1;
            }
        }
        if (ch == -1) break; // upper common tangent
        if (side == -1) {
            for (it = head[nowl].begin(); it != head[nowl].end(); it++) {
                if (intersection(ptL, p[it->id], ptR, p[ch])) {
                    head[it->id].erase(it->c);
                    head[nowl].erase(it++);
                } else {
                    it++;
                }
            }
            nowl = ch;
            addEdge(nowl, nowr);
        } else {
            for (it = head[nowr].begin(); it != head[nowr].end(); it++) {
                if (intersection(ptR, p[it->id], ptL, p[ch])) {
                    head[it->id].erase(it->c);
                    head[nowr].erase(it++);
                } else {
                    it++;
                }
            }
            nowr = ch;
            addEdge(nowl, nowr);
        }
    }

    std::vector<std::pair<int, int>> getEdge() {
        std::vector<std::pair<int, int>> ret;
        ret.reserve(n);
        std::list<Edge>::iterator it;
        for (int i = 0; i < n; i++) {
            for (it = head[i].begin(); it != head[i].end(); it++) {
                if (it->id < i) continue;
                ret.push_back(std::make_pair(p[i].id, p[it->id].id));
            }
        }
        return ret;
    }
};
```

2.10. 二、三维计算几何

<https://oi-wiki.org/geometry/2d/>
<https://oi-wiki.org/geometry/3d/>

2.11. 仿射变换

3. 数论

3.1. Exgcd

```
i64 exgcd(i64 a, i64 b, i64&x, i64&y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    i64 g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
```

3.2. Bsgs (gcd(a,p)=1)

```
ll bsgs(ll a, ll b, ll p) {
    if (1 % p == b % p) return 0;
    unordered_map<ll, ll> hash;
    ll m = ceil(sqrt(p));

    // Baby-step: 预处理 a^j * b 并存入哈希表
    ll cur = b % p;
    for (int j = 0; j < m; j++) {
        hash[cur] = j;
        cur = cur * a % p;
    }

    // Giant-step: 计算 am = a^m, 然后枚举 i 检查 am^i
    ll am = 1;
    for (int i = 0; i < m; i++) am = am * a % p;

    cur = am;
    for (int i = 1; i <= m; i++) {
        if (hash.count(cur)) return i * m - hash[cur];
        cur = cur * am % p;
    }
    return -1; // 无解
}
```

3.3. ExCRT (扩展中国剩余定理)

```
// 这里的乘法取模是为了防止 (a * b) % m 溢出, 如果 a, b 较小可直接使用 a * b % m
ll qmul(ll a, ll b, ll m) {
    ll res = 0;
    while (b) {
        if (b & 1) res = (res + a) % m;
        a = (a + a) % m;
        b >>= 1;
    }
    return res;
}

// n 个方程: x = r[i] (mod m[i])
ll exCRT(vector<ll>&m, vector<ll>&r) {
    ll M = m[0], R = r[0]; // R 为当前的解, M 为当前的最小公倍数
    for (int i = 1; i < m.size(); i++) {
        ll x, y;
        ll d = exgcd(M, m[i], x, y);
        if ((r[i] - R) % d != 0) return -1; // 无解

        // 求解 M*x = r[i] - R (mod m[i])
        ll mod = m[i] / d;
        x = qmul((r[i] - R) / d, x, mod); // 这里的乘法注意溢出
        if (x < 0) x += mod;

        R += x * M;
        M = M / d * m[i];
        R = (R * M + M) % M;
    }
    return R;
}
```

3.4. 拓展欧拉定理

$$a^k \equiv \begin{cases} a^{k \bmod \varphi(m)} & \gcd(a, m) = 1 \\ a^k & \gcd(a, m) \neq 1 \wedge k < \varphi(m) \\ a^{k \bmod \varphi(m) + \varphi(m)} & \gcd(a, m) \neq 1 \wedge k \geq \varphi(m) \end{cases} \pmod{m}$$

4. 欧拉路径

```
const int MAXN = 100005;
struct Edge {
    int to, id;
};
vector<Edge> adj[MAXN];
int head[MAXN]; // 当前弧优化: 记录每个点处理到了第几条边
bool used[MAXN * 2]; // 标记边是否被访问过 (处理无向图或多重边)
vector<int> path; // 存储最终路径 (点序列或边序列)

void dfs(int u) {
    // 遍历从 u 出发的边, head[u] 保证每条边只被扫描一次
    for (int &i = head[u]; i < adj[u].size(); i++) {
        Edge e = adj[u][i]; // 注意这里必须先 i++ 再递归
        if (used[e.id]) continue;
        used[e.id] = true; // 标记该边已用
        dfs(e.to);
    }
    path.push_back(u); // 回溯时加入序列
}

// 得到路径: reverse(path.begin(), path.end());
```

5. 马拉车

```
// d[i] 表示以 i 为中心的最长回文半径
// 转换后的字符串长度为 2n+1, d[i]-1 即为原字符串中对应回文串的长度
vector<int> manacher(string s) {
    string t = "#";
    for (char c : s) t += c, t += "#";
    int n = t.size();
    vector<int> d(n);
    // r 为当前已知回文串覆盖的最右边界, l 为该边界对应的中心
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = (i > r) ? 1 : min(d[l + r - i], r - i + 1);
        while (0 <= i - k && i + k < n && t[i - k] == t[i + k]) k++;
        d[i] = k - 1;
        if (i + k > r) l = i - k, r = i + k;
    }
    return d;
}
```

6. 欧拉筛

```
const int MAXN = 1000005;
int primes[MAXN], cnt;
bool is_not_prime[MAXN];

void sieve(int n) {
    for (int i = 2; i <= n; i++) {
        if (!is_not_prime[i]) primes[cnt++] = i;
        for (int j = 0; j < cnt && i * primes[j] <= n; j++) {
            is_not_prime[i * primes[j]] = true;
            if (i % primes[j] == 0) break; // 核心: 保证每个合数只被最小质因子筛掉
        }
    }
}
```

7. 拉格朗日插值 (通用版, $O(n^2)$)

```
// n 个点 (x[i], y[i]), 求 f(k)
ll lagrange(int n, ll k, vector<ll>&x, vector<ll>&y) {
    ll ans = 0;
    for (int i = 0; i < n; i++) {
        ll up = y[i], down = 1;
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            up = up * (k - x[j]) % MOD + MOD % MOD;
            down = down * (x[i] - x[j]) % MOD + MOD % MOD;
        }
        ans = (ans + up * inv(down)) % MOD;
    }
    return ans;
}
```

8. 拉格朗日插值（连续整数）

```
ll pre[MAXN], suf[MAXN], fac[MAXN], inv_fac[MAXN];
// x_i 为 1, 2, ..., n 的情况, 求 f(k)
ll lagrange_linear(int n, ll k, vector<ll>&y) {
    if (k >= 1 && k <= n) return y[k - 1];

    pre[0] = suf[n + 1] = 1;
    for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] * (k - i % MOD + MOD) % MOD;
    for (int i = n; i >= 1; i--) suf[i] = suf[i + 1] * (k - i % MOD + MOD) % MOD;

    fac[0] = 1;
    for (int i = 1; i <= n; i++) fac[i] = fac[i - 1] * i % MOD;
    inv_fac[n] = inv(fac[n]);
    for (int i = n - 1; i >= 0; i--) inv_fac[i] = inv_fac[i + 1] * (i + 1) % MOD;

    ll ans = 0;
    for (int i = 1; i <= n; i++) {
        ll up = pre[i - 1] * suf[i + 1] % MOD * y[i - 1] % MOD;
        ll down = inv_fac[i - 1] * inv_fac[n - i] % MOD;
        // 分母符号取决于 (n-i) 的奇偶性
        if ((n - i) % 2) ans = (ans - up * down % MOD + MOD) % MOD;
        else ans = (ans + up * down % MOD) % MOD;
    }
    return ans;
}
```

9. Odt

```
struct Node {
    int l, r;
    mutable ll v; // mutable 允许在 set 中直接修改 v
    Node(int L, int R = -1, ll V = 0) : l(L), r(R), v(V) {}
    bool operator<(const Node& t) const { return l < t.l; }
};

set<Node> odt;

// 分裂操作: 将包含 pos 的区间 [l, r] 分裂为 [l, pos-1] 和 [pos, r]
// 返回后一个区间的迭代器
auto split(int pos) {
    auto it = odt.lower_bound(Node(pos));
    if (it != odt.end() && it->l == pos) return it;
    --it;
    int l = it->l, r = it->r;
    ll v = it->v;
    odt.erase(it);
    odt.insert(Node(l, pos - 1, v));
    return odt.insert(Node(pos, r, v)).first;
}

// 核心: 区间赋值 (推平操作)
void assign(int l, int r, ll v) {
    auto itR = split(r + 1), itL = split(l); // 顺序必须先 R 后 L
    odt.erase(itL, itR);
    odt.insert(Node(l, r, v));
}

// 其他区间操作模板 (以区间加为例)
void add(int l, int r, ll v) {
    auto itR = split(r + 1), itL = split(l);
    for (; itL != itR; ++itL) itL->v += v;
}
```

10. 前缀数组（来自 oiwiki）

```
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}
```

11. kmp（来自 oiwiki）

```
vector<int> find_occurrences(string text, string pattern) {
    string cur = pattern + '#' + text;
    int sz1 = text.size(), sz2 = pattern.size();
    vector<int> v;
    vector<int> lps = prefix_function(cur);
    for (int i = sz2 + 1; i <= sz1 + sz2; i++) {
        if (lps[i] == sz2) v.push_back(i - 2 * sz2);
    }
}
```

```
return v;
}
```

12. mt19937

```
#include<bits/stdc++.h>
int main() {
    std::random_device rd;
    std::mt19937 gen(rd());

    std::uniform_int_distribution<>dis(1, 100);
    //std::uniform_real_distribution<double>dis(0.0, 1.0);
    for (int i = 0; i < 5; ++i) {
        std::cout << dis(gen) << " ";
    }

    return 0;
}
```

13. 二分图最大匹配（匈牙利算法，稠密图且点数大于 1e4 不适用）

```
const int MAXN = 505; // 左侧集合点的最大数量
vector<int> adj[MAXN]; // 邻接表, 只存从左往右的边
int match[MAXN]; // match[y] = x 表示右侧点 y 匹配了左侧点 x
bool vis[MAXN]; // 标记右侧点在单次 DFS 中是否被访问过
bool dfs(int u) {
    for (int v : adj[u]) {
        if (!vis[v]) {
            vis[v] = true;
            // 如果右侧点没有匹配, 或者原匹配点可以找到新的增广路
            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}

int solve(int n_left) {
    int ans = 0;
    memset(match, -1, sizeof(match));
    for (int i = 1; i <= n_left; i++) {
        memset(vis, false, sizeof(vis)); // 每次都要重置
        if (dfs(i)) ans++;
    }
    return ans;
}
```

14. 线性基

```
typedef long long ll;
const int MAXL = 60;
struct LinearBasis {
    ll a[MAXL + 1];

    LinearBasis() {
        memset(a, 0, sizeof(a));
    }

    // 简洁插入 (不消元, 只保证线性无关)
    void insert(ll x) {
        for (int i = MAXL; i >= 0; --i) {
            if ((x >> i & 1)) continue;
            if (!a[i]) { a[i] = x; return; }
            x ^= a[i];
        }
    }

    // 查询异或最大值 (贪心即可, 不需要消元)
    ll queryMax() {
        ll res = 0;
        for (int i = MAXL; i >= 0; --i) {
            if ((res ^ a[i]) > res) res ^= a[i];
        }
        return res;
    }

    // 合并另一个线性基
    void merge(const LinearBasis& other) {
        for (int i = 0; i <= MAXL; ++i) {
            if (other.a[i]) insert(other.a[i]);
        }
    };

    int main() {
        int n;
        cin >> n;
        LinearBasis lb;
    }
}
```

```
for (int i = 0; i < n; ++i) {
    ll x;
    cin >> x;
    lb.insert(x);
}
cout << lb.queryMax() << "\n";
return 0;
```

15. 强连通分量

```
const int N = 1e5 + 5;
vector<int> g[N];
int dfn[N], low[N], timer;
bool inStk[N];
vector<int> stk; // 模拟栈
vector<vector<int>> scc; // 收集所有强连通分量

void tarjan(int u) {
    dfn[u] = low[u] = ++timer;
    stk.push_back(u);
    inStk[u] = true;
    for (int v : g[u]) {
        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (inStk[v]) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] == low[u]) { // u 是 SCC 的根
        vector<int> comp;
        while (true) {
            int v = stk.back();
            stk.pop_back();
            inStk[v] = false;
            comp.push_back(v);
            if (v == u) break;
        }
        scc.push_back(comp); // 收集当前分量
    }
}

int main() {
    read(n, m, g); // g 为 edge
    for (int i = 1; i <= n; ++i)
        if (!dfn[i]) tarjan(i);
    // 输出所有强连通分量
    for (int i = 0; i < (int)scc.size(); ++i) {
        cout << "SCC " << i + 1 << ": ";
        for (int x : scc[i]) cout << x << " ";
        cout << "\n";
    }
    return 0;
```

16. polya 定理

$$|\frac{C}{G}| = \frac{1}{|G|} \sum_{g \in G} m^{c(g)}$$

- 置换群
- 群的阶（置换个数）
- 所有着色方案（共 m^n 种）
- 轨道集合（本质不同着色）
- 可用颜色数
- 置换 g 的循环个数
- 在 g 下保持不变的着色数

$$|\frac{X}{G}| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

补充：burnside 引理

- 被作用的集合
- 轨道集
- 群阶
- g 作用下的不动点集

17. 求割点

```
const int N = 1e5 + 5;
vector<int> g[N];
int dfn[N], low[N], timer;
bool cut[N]; // cut[u] = true 表示 u 是割点

void tarjan(int u, int fa) {
    dfn[u] = low[u] = ++timer;
    int child = 0; // 子树个数（仅对根有用）
    for (int v : g[u]) {
        if (v == fa) continue;
        if (!dfn[v]) {
            child++;
            tarjan(v, u);
        }
    }
}
```

```
low[u] = min(low[u], low[v]);
if (low[v] >= dfn[u]) cut[u] = true; // 割点判定
} else {
    low[u] = min(low[u], dfn[v]);
}
}
if (fa == -1) cut[u] = (child >= 2); // 根节点特判
}
// 调用：
// for (int i = 1; i <= n; i++)
//     if (!dfn[i]) tarjan(i, -1);
```

18. 自定义 hash

```
struct Person {
    std::string name;
    int age;
};
// 必须在 std 中打开 namespace
namespace std {
    template<>
    struct hash<Person> {
        size_t operator()(const Person& p) const noexcept {
            // 组合多个成员的哈希值（常用方法见下文）
            size_t h1 = hash<string>{}(p.name);
            size_t h2 = hash<int>{}(p.age);
            return h1 ^ (h2 << 1); // 简单组合，更稳健的方法见下
        }
    };
}
```

```
namespace std {
    template<typename T1, typename T2>
    struct hash<pair<T1, T2>> {
        size_t operator()(const pair<T1, T2>& p) const {
            auto h1 = hash<T1>{}(p.first);
            auto h2 = hash<T2>{}(p.second);
            // 组合方法...
        }
    };
}
```

```
struct Point {
    int x, y;
};
namespace std {
    template<>
    struct hash<Point> {
        size_t operator()(const Point& p) const {
            size_t seed = 0;
            hash<int> hasher;
            // 对每个字段依次调用 hash_combine
            hash_combine(seed, hasher(p.x));
            hash_combine(seed, hasher(p.y));
            return seed;
        }
    };
}
inline void hash_combine(size_t& seed, size_t hash) {
    seed ^= hash + 0x9e3779b97f4a7c15ULL + (seed << 6) + (seed >> 2);
}
```

19. hall 定理

- 设二分图 $G = (L, R, E)$, L 为左部, R 为右部。
- 1) 存在匹配覆盖 L 中所有顶点 当且仅当 对任意子集 $S \subseteq L$, 其邻域 $N(S)$ 满足:
 - $|N(S)| \geq |S|$
- 2) 最大匹配大小 = $|L| - \max_{S \subseteq L} \{|S| - |N(S)|\}$
- (若该值 < $|L|$, 则无法完全覆盖 L)

20. Zobrist Hashing (xor hashing)

赋值: 每个值 $v \rightarrow$ 随机 64 位整数 $hash[v]$

前缀: $pre[i] = pre[i-1] \oplus hash[A[i]]$

比较: 子数组 $XOR = pre[r] \oplus pre[l-1]$, 与预期哈希比较

多测放全局开 `mt19937`

```
//mt19937 rng32(...); // 生成 uint32_t
//mt19937_64 rng64(...); // 生成 uint64_t
```

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
hash[i] = rng();
```

21. Lemma (Bertrand's postulate)

For each positive integer x , there is a prime p inside the interval $[x, 2x]$.

22. 求多个数的欧拉函数

```
vector<int> cal_euler(int x) {
    vector<int> re(x + 1);
    vector<int> prime;
    vector<int> not_prime(x + 1, 0);
    for (int i = 2; i <= x; i++) {
        if (!not_prime[i]) {
            prime.emplace_back(i);
            re[i] = i - 1;
        }
        for (auto j : prime) {
            if (i * j > x) break;
            not_prime[i * j] = 1;
            if (i % j == 0) {
                re[i * j] = re[i] * j;
            } else {
                re[i * j] = re[i] * re[j];
            }
            if (i % j == 0) break;
        }
    }
    return re;
}
```

23. 最大流 (dinic 算法)

```
struct Dinic {
    struct Edge { int to, rev; long long cap; };
    vector<vector<Edge>> g;
    vector<int> lev, iter;
    void init(int n) { g.assign(n, {}); }
    void add_edge(int u, int v, long long cap) {
        g[u].push_back({v, (int)g[v].size(), cap});
        g[v].push_back({u, (int)g[u].size() - 1, 0});
    }
    void bfs(int s) {
        fill(lev.begin(), lev.end(), -1);
        queue<int> q;
        lev[s] = 0; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto &e : g[u]) {
                if (e.cap > 0 && lev[e.to] == -1) {
                    lev[e.to] = lev[u] + 1;
                    q.push(e.to);
                }
            }
        }
    }
    long long dfs(int u, int t, long long f) {
        if (u == t) return f;
        for (int &i = iter[u]; i < (int)g[u].size(); ++i) {
            auto &e = g[u][i];
            if (e.cap > 0 && lev[e.to] == lev[u] + 1) {
                long long d = dfs(e.to, t, min(f, e.cap));
                if (d > 0) {
                    e.cap -= d;
                    g[e.to][e.rev].cap += d;
                    return d;
                }
            }
        }
        return 0;
    }
    long long max_flow(int s, int t) {
        long long flow = 0;
        lev.resize(g.size()); iter.resize(g.size());
        while (true) {
            bfs(s);
            if (lev[t] == -1) break;
            fill(iter.begin(), iter.end(), 0);
            long long f;
            while ((f = dfs(s, t, 1e18)) > 0) flow += f;
        }
        return flow;
    }
};
```

最小割：最大流 = 最小割。求完最大流后，从源点沿剩余容量 > 0 的边 DFS，能到达的点集就是 S ，其余为 T 。

24. 有源汇最大流

```
struct BoundFlow {
    Dinic dinic;
    vector<long long> in;
    int S, T; // 超级源汇
    void init(int n) {
        dinic.init(n + 2);
        in.assign(n + 2, 0);
        S = n; T = n + 1;
    }
    // 添加有上下界的边 (u->v, l, r)
    void add_edge(int u, int v, long long l, long long r) {
        in[u] -= l; in[v] += l;
        dinic.add_edge(u, v, r - l);
    }
    bool feasible() {
        long long total = 0;
        for (int i = 0; i < S; ++i) { // S 是原图中最后一个点下标+1?
            if (in[i] > 0) {
                dinic.add_edge(S, i, in[i]);
                total += in[i];
            } else if (in[i] < 0) {
                dinic.add_edge(i, T, -in[i]);
            }
        }
        return dinic.max_flow(S, T) == total;
    }
    // 有源汇 s,t 的最大流 (先调用 add_edge 建图再调这个)
    long long max_flow_with_bound(int s, int t) {
        add_edge(t, s, 0, 1e18); // 加下限 0 的边
        if (!feasible()) return -1; // 无解
        long long flow = dinic.g[s].back().cap; // t->s 反向边容量 = 可行流中 s->t 的流量
        // 删掉 t->s 附加边
        dinic.g[t].pop_back(); dinic.g[s].pop_back();
        return flow + dinic.max_flow(s, t);
    }
};
```

上面 `feasible()` 中的 S 是 S 的点编号，初始化时 $S = n$ ，但循环里 $i < S$ 跑到了 $n-1$ ，即所有原图点 $(0-n-1)$ 。初始化时传入点数即可。

无源汇可行流
每个点计算 $\text{in}[v] = \text{所有入边下限和} - \text{出边下限和}$ 。
建超级源 S 和超级汇 T 。
若 $\text{in}[v] > 0$ ，连 $S \rightarrow v$ 容量 $\text{in}[v]$ ；若 $\text{in}[v] < 0$ ，连 $v \rightarrow T$ 容量 $-\text{in}[v]$ 。
原边连容量 $r - l$ 。
跑最大流，若从 S 出发的所有边满流，则有解。

25. 最小费用最大流 (spfa)

```
struct MCMF {
    struct Edge { int to, rev; long long cap, cost; };
    vector<vector<Edge>> g;
    vector<long long> dist;
    vector<int> pre, pre_id;
    vector<bool> inq;
    void init(int n) { g.assign(n, {}); }
    void add_edge(int u, int v, long long cap, long long cost) {
        g[u].push_back({v, (int)g[v].size(), cap, cost});
        g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
    }
    bool spfa(int s, int t) {
        dist.assign(g.size(), 1e18);
        inq.assign(g.size(), false);
        pre.assign(g.size(), -1);
        pre_id.assign(g.size(), -1);
        queue<int> q;
        dist[s] = 0; q.push(s); inq[s] = true;
        while (!q.empty()) {
            int u = q.front(); q.pop(); inq[u] = false;
            for (int i = 0; i < (int)g[u].size(); ++i) {
                auto &e = g[u][i];
                if (e.cap > 0 && dist[e.to] > dist[u] + e.cost) {
                    dist[e.to] = dist[u] + e.cost;
                    pre[e.to] = u;
                    pre_id[e.to] = i;
                    if (!inq[e.to]) {
                        q.push(e.to);
                        inq[e.to] = true;
                    }
                }
            }
        }
        return dist[t] != 1e18;
    }
    pair<long long, long long> min_cost_flow(int s, int t) {
        long long flow = 0, cost = 0;
        while (spfa(s, t)) {
            long long f = 1e18;
            for (int v = t; v != s; v = pre[v])
                f = min(f, g[pre[v]][pre_id[v]].cap);
            flow += f;
            for (int v = t; v != s; v = pre[v]) {
                g[v].back().cap -= f;
                g[pre[v]][pre_id[v]].cap += f;
                cost += f * g[v].back().cost;
            }
        }
        return {flow, cost};
    }
};
```

```

    auto &e = g[pre[v]][pre_id[v]];
    e.cap -= f;
    g[v][e.rev].cap += f;
    cost += f * e.cost;
}
}
return {flow, cost};
};

```

如果费用非负且追求稳定复杂度，可用 Dijkstra + 势能 代替 SPFA。

```

    int nsum = (sum + d) % D; // 状态更新按题目改
    res = (res + dfs(pos + 1, nsum, ntight, false)) % MOD;
}
}
return memo[pos][sum][tight][leadZero] = res;
}

```

26. 开区间二分(l,r)

它的核心思想可以总结为“红蓝染色法”（维护不变量）。

💡 核心法则：我们将数组分为两半，一半是不满足条件的（红色，设为 l），一半是满足条件的（蓝色，设为 r）。

l 始终指向“不满足条件”的位置。r 始终指向“满足条件”的位置。

1. 初始化：因为 l 必须一开始就不满足条件，r 必须一开始就满足条件，我们要把它们放在数组界外（或者已知的边界上）。对于一个长度为 m 的数组 b（下标 0 - m - 1）

int l = -1; // -1 绝对不满足条件（越界了，算作不满足）

int r = m; // m 绝对满足条件（我们要找的答案一定在 m 左侧，所以 m 兜底）

2. 循环条件：既然是开区间 (l, r)，说明 l 和 r 之间还有元素。只要它们俩之间还有至少一个元素，就继续二分

while (l + 1 < r) // 当 l 和 r 相邻时（比如 l=2, r=3），循环结束

3. 状态转移（极度舒适，不需要 +1 或 -1）

int mid = l + (r - l) / 2;

if (check(mid) == true) {

 r = mid; // mid 满足条件，所以把“蓝边界”r 移到 mid

} else {

 l = mid; // mid 不满足条件，所以把“红边界”l 移到 mid

}

4. 退出循环时的特点（重点！）：退出时，必定有 l + 1 == r（即 l 和 r 紧紧挨在一起）。因为 l 始终是不满足条件的最后一个位置，r 始终是满足条件的第一个位置。所以，你要找的目标答案就是 r！根本不用额外去推导什么边界。

```

int gcd(int a, int b) {
    int az = __builtin_ctz(a);
    int bz = __builtin_ctz(b);
    int z = min(az, bz);
    b >>= bz;
    while (a) {
        a >>= az;
        int diff = a - b;
        az = __builtin_ctz(diff);
        b = min(a, b), a = abs(diff);
    }
    return b << z;
}

```

```

vector<array<int,3>> cal(int x) {
    vector<array<int,3>> re(x + 1, {1, 1, 1});
    vector<int> not_prime(x + 1, 0);
    vector<int> prime;
    prime.reserve(x / 10);
    for (int i = 2; i <= x; i++) {
        if (!not_prime[i]) {
            prime.emplace_back(i);
            re[i] = {1, 1, i};
        }
        for (int j : prime) {
            if (i * j > x) break;
            not_prime[i * j] = 1;
            auto &[a, b, c] = re[i];
            int nj = a * j;
            if (nj <= b) {
                re[i * j] = {nj, b, c};
            } else if (nj <= c) {
                re[i * j] = {b, nj, c};
            } else {
                re[i * j] = {b, c, nj};
            }
            if (i % j == 0) break;
        }
    }
    return re;
}

```

```

int fast_gcd(int a, int b) {
    if (!a || !b) return (a | b);
    int az = __builtin_ctz(a);
    int bz = __builtin_ctz(b);
    int z = min(az, bz);
    b >>= bz;
    while (a) {
        a >>= az;
        int diff = a - b;
        az = __builtin_ctz(diff);
        b = min(a, b);
        a = abs(diff);
    }
    return b << z;
}

vector<vector<int>> get_gcd(int x) {
    int up = 1e3;
    vector<vector<int>> re(up + 1, vector<int>(up + 1));
    for (int i = 1; i <= up; i++) {
        for (int j = 1; j <= i; j++) {
            re[i][j] = re[j][i] = fast_gcd(i, j);
        }
    }
    for (int i = 0; i <= up; i++) {
        re[0][i] = re[i][0] = i;
    }
    return re;
}

/*
    auto &[x, y, z] = fact[b[j]];
    int _i = a[i];
    int gcd;
    if (x > 1) {
        gcd = _gcd[_i % x][x];
        tmp = tmp * gcd % mod;
        _i /= gcd;
    }

    if (y > 1) {
        gcd = _gcd[_i % y][y];
        tmp = tmp * gcd % mod;
    }
}

```

27. 线性求逆元

```

inv_num[1] = 1;
for (int i = 2; i < limit; i++) {
    inv_num[i] = (mod - mod / i) * inv_num[mod % i] % mod;
}

```

28. 数位 dp

```

string S; // 上限
int n = S.size();
// dp[pos][state][tight]
// state 按题目定义（余数/和/积等）
vector<vector<vector<int>>> dp(n + 1, vector<vector<int>>(MAX_STATE,
vector<int>(2, 0)));
dp[0][初始状态][1] = 1; // 第0位，初始状态，受限
for (int i = 0; i < n; i++) {
    for (int st = 0; st < MAX_STATE; st++) {
        for (int tight = 0; tight < 2; tight++) {
            if (dp[i][st][tight] == 0) continue;

            int limit = tight ? (S[i] - '0') : 9;
            for (int d = 0; d <= limit; d++) {
                int nst = 转移(st, d); // 按题目改
                int ntight = tight && (d == limit);
                dp[i + 1][nst][ntight] = (dp[i + 1][nst][ntight] + dp[i][st][tight]) % MOD;
            }
        }
    }
}

// 最终答案：所有 tight 状态的和
for (int tight = 0; tight < 2; tight++) {
    ans = (ans + dp[n][目标状态][tight]) % MOD;
}

string S; // 上限数字的字符串
int D; // 题目参数
int n;
int memo[10005][105][2][2]; // 按题目需求调整大小
int dfs(int pos, int sum, bool tight, bool leadZero) {
    if (pos == n) {
        return sum == 0 ? 1 : 0; // 终止条件按题目改
    }
    if (memo[pos][sum][tight][leadZero] != -1) return memo[pos][sum][tight][leadZero];
    int res = 0;
    int limit = tight ? (S[pos] - '0') : 9;
    for (int d = 0; d <= limit; d++) {
        bool ntight = tight && (d == limit);
        bool nleadZero = leadZero && (d == 0);

        if (leadZero && d == 0) {
            res = (res + dfs(pos + 1, sum, ntight, true)) % MOD; // 前导零，不贡献
        } else {

```

```

        _i /= gcd;
    }

    if (z > 1e3) {
        tmp = tmp * (_i % z == 0 ? z : 1) % mod;
    }
    else {
        gcd = _gcd[_i % z][z];
        tmp = tmp * gcd % mod;
    }
}

*/

```

```

friend ostream& operator<<(ostream& out, const BigInt& x) {
    out << x.s.back();
    for (int i = x.s.size() - 2; i >= 0; --i) {
        out << setfill('0') << setw(WIDTH) << x.s[i];
    }
    return out;
};

```

(2) 版本二（较短，功能简单）

```

struct Big {
    vector<int> v;
    const int B = 1e9;

    Big() {} // 默认空向量
    Big(long long n) { do { v.push_back(n % B); n /= B; } while (n); }
    Big(string s) {
        // 利用 max/min 优雅避开边界 if/else 截取判定
        for (int i = s.size(); i > 0; i -= 9)
            v.push_back(stoi(s.substr(max(0, i - 9), min(i, 9))));
        trim();
    }

    void trim() { while (v.size() > 1 && v.back() == 0) v.pop_back(); }

    Big operator+(const Big &b) const {
        Big c;
        // 进位 t 融合进循环条件，省去最后的 if(t) 判定
        for (int i = 0, t = 0; i < v.size() || i < b.v.size() || t; ++i) {
            if (i < v.size()) t += v[i];
            if (i < b.v.size()) t += b.v[i];
            c.v.push_back(t % B);
            t /= B;
        }
        return c;
    }

    Big operator-(const Big &b) const {
        Big c = *this;
        for (int i = 0, t = 0; i < c.v.size(); ++i) {
            t = c.v[i] - t - (i < b.v.size() ? b.v[i] : 0);
            c.v[i] = (t + B) % B;
            t = t < 0; // 核心: t < 0 时自动转为 1 形成借位，否则为 0
        }
        c.trim(); return c;
    }

    Big operator*(const Big &b) const {
        Big c; c.v.assign(v.size() + b.v.size(), 0);
        for (int i = 0; i < v.size(); ++i) {
            long long t = 0;
            // 乘法的进位累加直接一气呵成
            for (int j = 0; j < b.v.size() || t; ++j) {
                t += c.v[i + j] + 1ll * v[i] * (j < b.v.size() ? b.v[j] : 0);
                c.v[i + j] = t % B;
                t /= B;
            }
        }
        c.trim(); return c;
    }

    bool operator<(const Big &b) const {
        if (v.size() != b.v.size()) return v.size() < b.v.size();
        for (int i = v.size() - 1; i >= 0; --i)
            if (v[i] != b.v[i]) return v[i] < b.v[i];
        return false;
    }

    friend ostream& operator<<(ostream& os, const Big& a) {
        if (a.v.empty()) return os << 0;
        os << a.v.back();
        for (int i = a.v.size() - 2; i >= 0; --i)
            os << setfill('0') << setw(9) << a.v[i];
        return os;
    }
};

```

31. 闭区间线段树

主要是文字说明一些注意点，比如左右区间为 $[l, m]$ 和 $[m, r]$ ， $tl < m$ ，走左， $tr > m$ 走右，点是没有长度的，长度由两个点相减得到，叶子节点 $r-l=1$ ， $r-l=1$ 才算有效区间，在一些区间问题常用这个写法，比如线段长度

30. 高精度

(1) 版本一（长功能多）

```

struct BigInt {
    static const int BASE = 1e9;
    static const int WIDTH = 9;
    vector<int> s;

    BigInt(long long num = 0) { *this = num; }
    BigInt(string str) { *this = str; }

    BigInt& operator=(long long num) {
        s.clear();
        do { s.push_back(num % BASE); num /= BASE; } while (num > 0);
        return *this;
    }

    BigInt& operator=(const string& str) {
        s.clear();
        for (int i = str.length(); i > 0; i -= WIDTH) {
            if (i < WIDTH) s.push_back(stoi(str.substr(0, i)));
            else s.push_back(stoi(str.substr(i - WIDTH, WIDTH)));
        }
        trim();
        return *this;
    }

    void trim() {
        while (s.size() > 1 && s.back() == 0) s.pop_back();
    }

    BigInt operator+(const BigInt& b) const {
        BigInt c; c.s.clear();
        for (int i = 0, carry = 0; i < s.size() || i < b.s.size() || carry; ++i) {
            if (i < s.size()) carry += s[i];
            if (i < b.s.size()) carry += b.s[i];
            c.s.push_back(carry % BASE);
            carry /= BASE;
        }
        return c;
    }

    BigInt operator-(const BigInt& b) const {
        BigInt c = *this;
        for (int i = 0, borrow = 0; i < c.s.size(); ++i) {
            borrow = c.s[i] - borrow - (i < b.s.size() ? b.s[i] : 0);
            c.s[i] = borrow < 0 ? borrow + BASE : borrow;
            borrow = borrow < 0 ? 1 : 0;
        }
        c.trim();
        return c;
    }

    BigInt operator*(const BigInt& b) const {
        BigInt c; c.s.assign(s.size() + b.s.size(), 0);
        for (int i = 0; i < s.size(); ++i) {
            long long carry = 0;
            for (int j = 0; j < b.s.size() || carry; ++j) {
                long long cur = c.s[i + j] + s[i] * 1ll * (j < b.s.size() ? b.s[j] : 0) + carry;
                c.s[i + j] = cur % BASE;
                carry = cur / BASE;
            }
        }
        c.trim();
        return c;
    }

    bool operator<(const BigInt& b) const {
        if (s.size() != b.s.size()) return s.size() < b.s.size();
        for (int i = s.size() - 1; i >= 0; --i)
            if (s[i] != b.s[i]) return s[i] < b.s[i];
        return false;
    }

    bool operator==(const BigInt& b) const {
        return !(*this < b) && !(b < *this);
    }

    friend istream& operator>>(istream& in, BigInt& x) {
        string str; if (in >> str) x = str; return in;
    }
};

```

32. 杂项

32.1. sos dp

```
/*
    做n次一维前缀和
*/
for(int i[1]=1;i[1]<=n;i[1]++){
    for(int i[2]=1;i[2]<=n;i[2]++){
        ...
        for(int i[n]=1;i[n]<=n;i[n]++){
            a[i[1]][i[2]][i[3]]...+=a[i[1]-1][i[2]]...;
        }
    }
}
for(int i[1]=1;i[1]<=n;i[1]++){
    for(int i[2]=1;i[2]<=n;i[2]++){
        ...
        for(int i[n]=1;i[n]<=n;i[n]++){
            a[i[1]][i[2]][i[3]]...+=a[i[1]][i[2]-1]...;
        }
    }
}
```

1.upper_bound和lower_bound比map更快

待施工：fwt，博弈论，根号分治，调和级数，点分治，polya定理带权重的版本，猫树，无旋 treap，splay 树，区间 gcd 最多下降 $\log$ 次，斐波那契数列的性质应用（每项大等于前一项，每一项小等于前一项的两倍，每一项等于前两项的和），重心点分治每次规模除二，st 表二分，对顶堆，pbds，isap 被特意卡的话常数比 dinic 大，

// 1. $(x \& b) > (b > 1)$ 判断 x 在 b 的最高位是否为 1

// $a + b == (a \wedge b) + 2 * (a \& b)$

std::set: 通过 Key、Compare、Allocator 三个参数，定义键类型、排序规则和内存分配。

std::map: 在 set 的基础上增加 T 值类型，形成键、值、比较器、分配器四个可调参数。

std::unordered_set: 用 Hash、KeyEqual 替代 Compare，组合成键、哈希、判等、分配器四个参数的无序集合。

std::unordered_map: 在 unordered_set 模板参数上增加 T 值类型，成为键、值、哈希、判等、分配器五个可调参数的无序映射。

Bfs trick : Problem - B - Codeforces (来自 cc cpc)

四个边框加入队列中 bfs，可方便解决左上角和右下角是否连通等问题（本题左边框和下边框作为起点 bfs，扫八个方向，标记为 1，右边框与上边框作为起点，扫八个方向，标记为 2，如果 1 与 2 连接（第二次八方扫描时扫描到），则左上与右下被隔绝）

当前树平均深度为 A_m ，则加入一点作为原图上点的新儿子的期望深度为 $A_m + 1$ ，有时候期望计算可以从平均角度考虑

二进制分组

根据经典结论全零序列区间 得到序列 需要花费

数与数之间的关系问题转化为图上问题

矩阵乘法优化 dp 问题

lcp 优化--Ukkonen 算法或 Myers 差分算法

期望 dp，计数 dp（插入 dp），动态 dp，状压 dp

Sub = (sub - 1) & mask // 遍历该集合的所有子集

笛卡尔树，差分约束，猫树，折半搜索，判断能否构成立体图形计算某点所有距离平方